

Name: Aziz Sayyad

Roll No: 381069

Batch: A3

Assignment No: 8

Problem Statement:

Apply WordNet-based Word Sense Disambiguation to improve meaning interpretation in ambiguous sentences.

Objective:

- To understand the problem of lexical ambiguity and the need for Word Sense Disambiguation (WSD) in NLP.
- To study the structure of WordNet synsets and how they represent distinct word senses.
- To implement the Lesk algorithm for unsupervised WSD using NLTK's built-in lesk() function.
- To apply WSD on ambiguous sentences and retrieve the predicted synset and its definition.
- To analyze how contextual tokens influence the Lesk algorithm's sense selection for ambiguous words.

S/W Packages and H/W apparatus used:

- Operating System: Windows/Linux/macOS
- Kernel: Python 3.x
- Tools: Google Colab / Jupyter Notebook
- Hardware: CPU with minimum 4GB RAM
- Libraries and packages used: NLTK (punkt, punkt_tab, wordnet, omw-1.4), nltk.wsd (lesk), nltk.corpus (wordnet)

Theory:

Lexical ambiguity occurs when a single word has multiple possible meanings depending on context. For example, the word 'bank' can refer to a financial institution, the side of a river, or tilting an aircraft. Natural language understanding systems must correctly resolve such ambiguities to process text accurately. Word Sense Disambiguation (WSD) is the NLP task of identifying which sense of an ambiguous word is intended in a given context.

WordNet and Synsets:

WordNet organizes English words into synsets (synonym sets), each representing a distinct concept or word sense. Every synset contains a definition, example sentences, and lexical relations. A polysemous word like 'bank' has multiple synsets -one for a financial institution (bank.n.01), another for a riverbank (bank.n.09), another for a savings container (savings_bank.n.02), and so on. WSD selects the most contextually appropriate synset for a given word occurrence.

The Lesk Algorithm:

The Lesk algorithm (1986) is the foundational unsupervised WSD method. It resolves ambiguity by comparing the words in the sentence context against the definitions (glosses) of each candidate synset in WordNet. The synset whose definition has the maximum word overlap with the sentence context is

selected as the correct sense. NLTK's `lesk(context_sentence, ambiguous_word)` implements this directly, accepting a list of context tokens and the target word, and returning the best-matching synset. The Simplified Lesk variant also considers examples and related synset glosses in the overlap computation.

Synset Definition Retrieval:

Once the Lesk algorithm returns a synset object, its definition can be retrieved by calling `sense.definition()`, which returns a human-readable string describing the meaning. The synset name follows the format `word.pos.sense_number` (e.g., `savings_bank.n.02`), where `pos` indicates the part of speech (`n` = noun, `v` = verb, `a` = adjective, `r` = adverb) and the sense number distinguishes multiple senses of the same word and POS.

Methodology:

- Install NLTK and download required corpora: `punkt`, `punkt_tab`, `wordnet`, and `omw-1.4`.
- Import `lesk` from `nltk.wsd`, `word_tokenize` from `nltk.tokenize`, and `wordnet` as `wn` from `nltk.corpus`.
- Define an ambiguous input sentence: 'I went to the bank to deposit money.' with the target ambiguous word 'bank'.
- Tokenize the sentence using `word_tokenize()` to produce a list of context tokens.
- Call `lesk(tokens, word)` to run the Lesk algorithm, which compares context token overlap against all WordNet synset definitions for 'bank'.
- Print the sentence, the ambiguous word, the predicted synset name, and its definition using `sense.definition()`.
- Analyze whether the predicted sense correctly matches the financial institution meaning intended by the sentence context.

Use Cases:

- Use Case 1 - Machine Translation: WSD disambiguates polysemous source words before translation, ensuring the correct target-language equivalent is chosen. For example, 'bank' in a financial context must translate differently from 'bank' as a riverbank in Hindi or any other language.
- Use Case 2 - Information Retrieval Enhancement: Search engines use WSD to interpret query words correctly, returning results relevant to the intended sense rather than all possible senses, improving precision for ambiguous queries like 'Java' (programming language vs. island).
- Use Case 3 - Question Answering Systems: WSD helps QA systems correctly interpret ambiguous words in questions and passages, preventing incorrect answer extraction caused by sense confusion in complex or domain-mixed text corpora.

Advantages:

- The Lesk algorithm requires no labeled training data, making it applicable to any domain or language covered by WordNet.
- NLTK's `lesk()` provides a simple, one-line API for WSD without requiring any model training or configuration.
- WordNet's rich synset definitions and examples provide detailed semantic context for disambiguation.
- WSD improves the accuracy of downstream NLP tasks such as translation, information retrieval, and summarization.

- The omw-1.4 extension enables multilingual WSD by linking WordNet synsets to equivalent senses in other languages.

Limitations:

- The basic Lesk algorithm has relatively low accuracy (~50-60%) due to sparse overlap between short sentences and synset definitions.
- Short sentences with few content words provide insufficient context for reliable disambiguation.
- WordNet does not cover proper nouns, technical jargon, neologisms, or informal language, limiting WSD coverage.
- The algorithm treats all context words equally without considering their proximity or grammatical relationship to the ambiguous word.
- Supervised WSD methods (e.g., using BERT embeddings) significantly outperform the Lesk algorithm but require annotated training corpora.

Applications:

- Natural language understanding: WSD enables chatbots and virtual assistants to correctly interpret ambiguous user queries.
- Sentiment analysis: Disambiguating adjectives like 'light' (weight vs. colour vs. brightness) improves opinion mining accuracy.
- Knowledge graph population: WSD links text mentions to the correct concept nodes in ontologies like DBpedia and Wikidata.
- Text-to-speech systems: WSD helps pronunciation engines select the correct phonetic form for heteronyms (words spelled alike but pronounced differently by sense).
- Semantic similarity computation: Comparing word senses rather than surface forms produces more meaningful text similarity scores for document clustering and recommendation systems.

Conclusion:

This assignment demonstrated WordNet-based Word Sense Disambiguation using NLTK's implementation of the Lesk algorithm. Applied to the ambiguous sentence 'I went to the bank to deposit money.', the algorithm tokenized the sentence, computed overlap between context words and WordNet synset definitions, and predicted the synset `savings_bank.n.02` -a container for keeping money at home - rather than the intended financial institution sense. This result highlights a known limitation of the basic Lesk algorithm: short context sentences with limited overlap often lead to incorrect sense selection. Nevertheless, the experiment effectively illustrates the WSD pipeline, the role of WordNet synsets in meaning representation, and the importance of richer contextual models for improving disambiguation accuracy in practical NLP systems.